

AI Camera — อ่านป้ายทะเบียนด้วย Raspberry Pi 5

อบรมเชิงปฏิบัติการ 2 วัน สำหรับนักเรียนระดับมัธยมศึกษาตอนปลาย

ระยะเวลา

2 วัน (09:00–16:30)

HARDWARE หลัก

Raspberry Pi 5 + Camera Module 3

เทคโนโลยี

Python · OpenCV · YOLOv8 · Tesseract OCR · Streamlit

 กล้อง (Capture)



 YOLOv8 (Crop)



 Tesseract (OCR)



 Match (77 จังหวัด)



 Streamlit GUI / Pipeline

วันที่ 1

พื้นฐานคอมพิวเตอร์ + Python + Hardware + Image Processing

🎯 ปูฐานราก SBC (RPi 5) + ไวยากรณ์ Python + Pi Camera Module 3 + การมองภาพด้วย OpenCV

เปิดคอร์ส + แนะนำโปรเจกต์ (Thai ANPR)

ทำความเข้าใจเป้าหมายของคอร์ส: สร้างระบบอ่านป้ายทะเบียนรถยนต์ภาษาไทยอัตโนมัติแบบครบวงจร

- 🎬 **Demo ระบบอ่านป้ายทะเบียนที่ทำงานจริง** — เห็นภาพรวมตั้งแต่ถ่ายภาพจนถึงได้ข้อความ
- ภาพรวม 2 วัน: จากศูนย์ → ระบบที่ตรวจจับป้าย อ่านตัวอักษรไทย/เลขทะเบียน และแมตช์ 77 จังหวัด
- เข้าใจความท้าทายของป้ายทะเบียนไทย: มี 2 บรรทัด, หมวดอักษร, เลขทะเบียน และชื่อจังหวัด
- ภาพรวม timeline: เข้าปูพื้นฐาน บ่ายลงมือทดลองจริงกับฮาร์ดแวร์



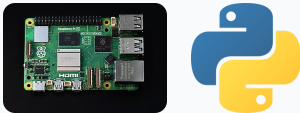
ป้ายทะเบียนรถยนต์ไทย — โจทย์หลักที่ระบบ AI Camera นี้จะเรียนรู้ให้ได้อ่าน (ภาพ: Wikimedia Commons)

Learning Roadmap (Part 1/2) – จากฐานรากสู่สายตาคอมพิวเตอร์

วันที่ 1: เริ่มต้นจากระบบฮาร์ดแวร์ ภาษา Python สู่การดึงภาพสดจากกล้องและการประมวลผลภาพขั้นต้น

Phase 1: ฐานรากอุปกรณ์ & ภาษาโปรแกรม

Part 1.1



01. Raspberry Pi 5 & Linux Terminal

Step 01

- 🎯 ทำทำไม: รู้จักระบบปฏิบัติการ Linux, คำสั่ง Terminal & สคริปต์ setup.sh
- ⚡ ช่วยอะไรได้: ควบคุมระบบ รันคำสั่ง จัดการ Environment และติดตั้ง dependencies

02. Python Core (basic_python)

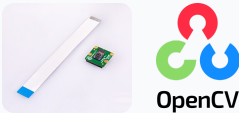
Step 02

- 🎯 ทำทำไม: เขียนตัวแปร เงื่อนไข if-else ลูป รายการ (List) และฟังก์ชันตรวจสอบ
- ⚡ ช่วยอะไรได้: เป็นภาษาหลักที่ใช้เขียน Logic และเชื่อมต่อทุกโมดูล AI เข้าด้วยกัน

💡 **ทำไมต้องเริ่มขั้นนี้?** เพื่อให้ทุกคนมีความพร้อมในการเขียนโค้ดและควบคุมบอร์ดได้อย่างมั่นใจ

Phase 2: สายตาคอมพิวเตอร์ & ภาพคลาสสิก

Part 1.2



03. Pi Camera Module 3 & picamera2

Step 03

- 🎯 ทำทำไม: ต่อสาย CSI, ถ่ายภาพด้วย capture.py และหน้าเว็บ capture_gui.py
- ⚡ ช่วยอะไรได้: ได้ภาพถ่ายความละเอียดสูงจากกล้องส่งตรงเข้าสู่ไพลเดอร์ระบบ

04. OpenCV & Contour Processing

Step 04

- 🎯 ทำทำไม: แปลงภาพ Grayscale, Resize, Canny Edge และหาขอบสี่เหลี่ยมด้วยกฎ
- ⚡ ช่วยอะไรได้: เข้าใจฟิสิกส์ภาพ และเห็นข้อจำกัดของวิธีเดิมก่อนนำ AI มาช่วย

💡 **ทำไมต้องขั้นนี้?** เพื่อให้เห็นจุดอ่อนของวิธีคลาสสิก (หลุดง่ายเมื่อแสง/มุมเปลี่ยน) ก่อนเริ่มใช้ AI

Learning Roadmap (Part 2/2) – จากโมเดล AI สู่ระบบอ่านป้ายจริง

วันที่ 2: นำ Deep Learning (YOLOv8) และ OCR มาตรวจจับ อ่านข้อความ ทำความสะอาดข้อมูล และสร้าง Web GUI

Phase 3: สมอง AI & การอ่านอักขระไทย

Part 2.1



05. YOLOv8n Detect & Crop

Step 05

- 🎯 ทำทำไม: โหลดโมเดล YOLOv8n ตรวจจับพิกัดป้ายทะเบียนและ Crop เฉพาะป้าย
- ⚡ ช่วยอะไรได้: ตัดเฉพาะบริเวณป้ายทะเบียนอย่างแม่นยำ ทนทานต่อแสงและมุมมอง

06. Thai Tesseract OCR

Step 06

- 🎯 ทำทำไม: Preprocess ขอบป้าย + แปลงขาวดำ + อ่านตัวหนังสือไทย/อังกฤษ
- ⚡ ช่วยอะไรได้: แปลงภาพป้ายทะเบียนเป็น Raw Text สำหรับนำไปประมวลผลต่อ

💡 **ทำไมต้องขั้นนี้?** แยกหน้าที่ "หาตำแหน่งป้าย" กับ "อ่านข้อความ" ออกจากกันเพื่อความแม่นยำสูงสุด

Phase 4: Data Matching, GUI & Full Pipeline

Part 2.2



07. Clean & Match 77 จังหวัด

Step 07

- 🎯 ทำทำไม: ใช้ Regex ตัดขยะ แยกเลขทะเบียน และ Fuzzy Match ชื่อ 77 จังหวัด
- ⚡ ช่วยอะไรได้: ได้ข้อมูลเลขทะเบียนและจังหวัดที่ถูกต้อง แม้อ OCR จะสะกดเพี้ยน

08. Streamlit Web GUI & Run Pipeline

Step 08

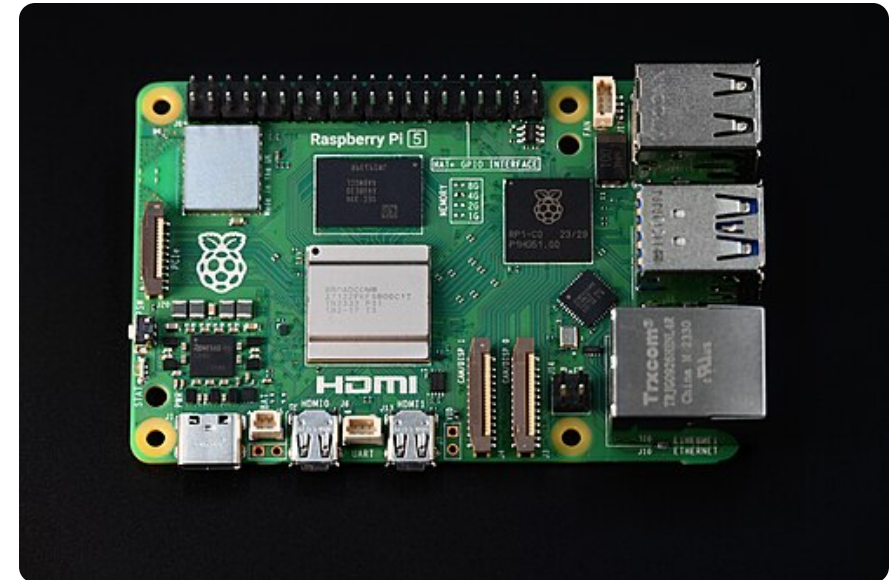
- 🎯 ทำทำไม: รวม 3 โมดูลผ่าน run_pipeline.py และสร้างหน้า Dashboard (app.py)
- ⚡ ช่วยอะไรได้: ได้ผลิตภัณฑ์ AI Camera สมบูรณ์แบบที่ผู้ใช้งานทั่วไปกดสั่งงานได้

💡 **ทำไมต้องขั้นนี้?** เปลี่ยนโค้ดแต่ละส่วนให้กลายเป็นระบบ End-to-End ที่พร้อมใช้งานจริง

ความเป็นมาของ Raspberry Pi เทียบกับ PC

ทำความเข้าใจคอมพิวเตอร์บอร์ดเดี่ยว (Single Board Computer) สำหรับงาน Embedded AI

- Raspberry Pi ถือกำเนิดขึ้นที่สหราชอาณาจักรเพื่อให้ทุกคนเข้าถึงคอมพิวเตอร์ราคาประหยัด
- **SBC (Single Board Computer)** รวม CPU, RAM, GPU, พอร์ตเชื่อมต่อไว้บนบอร์ดเดียวขนาดเท่าบัตรเครดิต
- กินพลังงานต่ำมาก (5V 5A) สามารถทำงาน 24/7 และใช้กับแบตเตอรี่ได้
- เหมาะอย่างยิ่งกับงาน **Edge AI, Smart Camera, IoT และระบบควบคุมอัตโนมัติ**

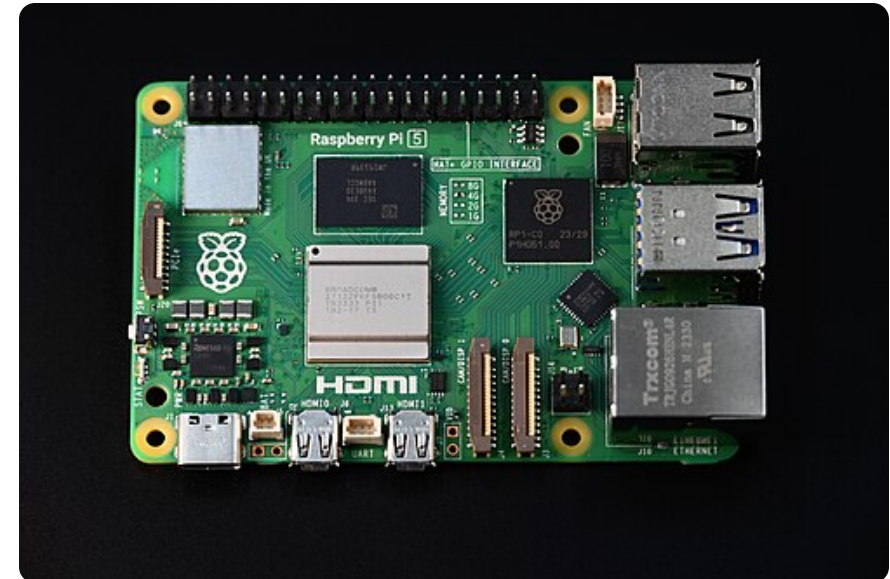


Raspberry Pi 5 — หัวใจหลักในการประมวลผลระบบ AI Camera ใน Workshop นี้

ส่วนประกอบพื้นฐานของ Raspberry Pi 5

เจาะลึกสเปกและฮาร์ดแวร์ที่ใช้ในการประมวลผลโมเดล AI

- **CPU** — Broadcom BCM2712 Quad-core 64-bit Arm Cortex-A76 @ 2.4GHz เร็วขึ้น 2-3 เท่า
- **RAM** — 4GB / 8GB LPDDR4X ส่งผ่านข้อมูลรวดเร็วสำหรับโหลดโมเดล AI
- **GPU** — VideoCore VII @ 800MHz รองรับ OpenGL ES 3.1, Vulkan 1.2
- **Camera Interface** — พอร์ต 2 x 4-lane MIPI CSI/DSI รองรับกล้องความเร็วสูง
- **I/O & Power** — PCIe 2.0, USB 3.0, Gigabit Ethernet, 40-Pin GPIO



Raspberry Pi 5 Layout — ชิปประมวลผล BCM2712 และพอร์ตเชื่อมต่อ CSI

รู้จัก Pi 5 ภาคปฏิบัติ & การติดตั้งโปรเจกต์

คำสั่ง Terminal พื้นฐาน และการใช้ setup.sh เพื่อเตรียมระบบอัตโนมัติ

- เปิด **Terminal** ลองคำสั่งพื้นฐาน: `ls`, `pwd`, `python3 --version`
- รับคำสั่ง `source setup.sh` เพื่อติดตั้งระบบ:
 - ติดตั้ง Tesseract OCR + ภาษาไทย (tesseract-ocr-tha)
 - ติดตั้ง python3-picamera2 สำหรับควบคุมกล้อง
 - สร้าง Python Virtual Environment (`--system-site-packages`)
 - ติดตั้ง PyTorch (CPU) และไลบรารีทั้งหมดใน `requirements.txt`

```
Terminal - setup.sh

pi@raspberrypi:~/ComVision_workshop $ source setup.sh
กำลังติดตั้ง Tesseract OCR ...
กำลังสร้าง venv (--system-site-packages) ...
เปิดใช้งาน venv แล้ว: /home/pi/.../venv
กำลังติดตั้ง torch (CPU-only) & requirements ...
✓ ติดตั้งเสร็จสมบูรณ์ พร้อมใช้งาน!
```

Python คืออะไร

ภาษาโปรแกรมมอดินิยมอันดับ 1 สำหรับงาน AI, Computer Vision และ Data Science

- ภาษาโปรแกรมที่อ่านง่าย เขียนสั้น ไวยากรณ์กระชับ ใกล้เคียงภาษาอังกฤษ
- **Code** คือชุดคำสั่งที่ละบรรทัดที่คอมพิวเตอร์จะประมวลผลจากบนลงล่าง
- มี Ecosystem ของไลบรารี AI ที่ทรงพลัง: OpenCV, PyTorch, Ultralytics, Streamlit
- เราจะเริ่มปูพื้นฐานจากไฟล์เดอร์ `basic_python/` ในโปรเจกต์

```
hello.py

# โปรแกรม Python บรรทัดแรกของเรา
print("Hello, Cytron AI Camera!")

# ผลลัพธ์ที่หน้าจอ Terminal
Hello, Cytron AI Camera!
```

ตัวแปร (Variable) และชนิดข้อมูล (Data Type)

ทำความเข้าใจการเก็บข้อมูลลงในหน่วยความจำผ่าน basic_python/01_variables_datatypes.py

- **ตัวแปร** คือ "กล่องเก็บข้อมูล" ที่มีชื่อกำกับ ใช้เครื่องหมาย = เพื่อกำหนดค่า
- **int** (จำนวนเต็ม) เช่น age = 16
- **float** (ทศนิยม) เช่น height = 168.5
- **str** (ข้อความ) เช่น name = "Got"
- **bool** (ตรรกะ) มีค่าเป็น True หรือ False

```
basic_python/01_variables_datatypes.py

# 01_variables_datatypes.py - ตัวแปรและชนิดข้อมูล
name = "Got"
age = 16
height = 168.5
is_student = True

print("สวัสดีคุณ", name)
print("อายุ:", age, "ส่วนสูง:", height)
print("เป็นนักเรียน:", is_student)
```

print() และ input() — ให้โปรแกรมคุยกับเรา

การติดต่อสื่อสารระหว่างผู้ใช้กับคอมพิวเตอร์ผ่าน Command Line

- **print()** ใช้แสดงผลลัพธ์ข้อความหรือค่าของตัวแปรออกทางหน้าจอ
- **input()** ใช้รับข้อความจากคีย์บอร์ดที่ผู้ใช้พิมพ์เข้ามา (ได้ผลลัพธ์เป็น **str** เสมอ)
- นำค่าที่รับมาไปแปลงชนิดข้อมูล (Type Casting) เช่น `int()` หรือ `float()` เพื่อคำนวณต่อได้

```
greet.py

name = input("ชื่อของคุณ: ")
plate_in = input("ระบุเลขทะเบียน: ")

print("สวัสดีคุณ", name)
print("บันทึกทะเบียน:", plate_in)

# ผู้ใช้พิมพ์: Got, 1กท1234
# ผลลัพธ์: สวัสดีคุณ Got | บันทึกทะเบียน: 1กท1234
```

Operators — การคำนวณและเปรียบเทียบ

สัญลักษณ์ทางคณิตศาสตร์และตรรกศาสตร์ที่ใช้ในการประมวลผล

- **คำนวณทางคณิตศาสตร์:** $+$ $-$ $*$ $/$, $//$ (หารปัดเศษ), $%$ (หารเอาเศษ)
- **เปรียบเทียบค่า:** $==$ (เท่ากับ), $!=$ (ไม่เท่ากับ), $>$, $<$, $>=$, $<=$
- ผลลัพธ์จากการเปรียบเทียบจะเป็นค่าความจริง True หรือ False เสมอ
- ใช้ร่วมกับ **if-else** เพื่อควบคุมทิศทางของโปรแกรม

```
operators.py

conf_score = 0.85
conf_threshold = 0.50

is_valid = conf_score >= conf_threshold
print("ผ่านเกณฑ์ความมั่นใจ:", is_valid)

# ผลลัพธ์: ผ่านเกณฑ์ความมั่นใจ: True
```

เงื่อนไข if-else

การตัดสินใจของโปรแกรมตามเงื่อนไขผ่าน basic_python/02_operators_ifelse.py

- เปรียบเสมือนทางแยกในการตัดสินใจว่าจะทำงานในบล็อกใด
- **if** ทำงานเมื่อเงื่อนไขเป็นจริง (True)
- **else** ทำงานเมื่อเงื่อนไขข้างบนไม่เป็นจริง
- **elif** ใช้ตรวจสอบเงื่อนไขเพิ่มเติมถัดไป

```
basic_python/02_operators_ifelse.py

# 02_operators_ifelse.py - การคำนวณและเงื่อนไข if-else
score = 75

if score ≥ 50:
    print("ผลการสอบ: ผ่านเกณฑ์ (PASS)")
else:
    print("ผลการสอบ: ไม่ผ่าน (FAIL)")
```

Loop (for / while) — การทำงานซ้ำอัตโนมัติ

ประมวลผลข้อมูลจำนวนมากอย่างรวดเร็วโดยไม่ต้องเขียนโค้ดซ้ำ

- **for loop** ใช้ทำซ้ำตามจำนวนรอบที่ทราบแน่นอน หรือวนอ่านสมาชิกในชุดข้อมูล
- **range(n)** สร้างลำดับตัวเลขตั้งแต่ 0 จนถึง n-1
- **while loop** ทำงานวนซ้ำตราบใดที่เงื่อนไขยังคงเป็นจริง (True)
- ใช้ในการประมวลผลภาพทีละเฟรมในวิดีโอ หรือวนตรวจจับภาพหลายๆ ไฟล์ในโฟลเดอร์

```
loop.py

# วนรอบประมวลผล 3 ครั้ง
for i in range(3):
    print("กำลังตรวจจับภาพที่", i + 1)

# กำลังตรวจจับภาพที่ 1
# กำลังตรวจจับภาพที่ 2
# กำลังตรวจจับภาพที่ 3
```

List — การเก็บและจัดการชุดข้อมูล

เก็บข้อมูลหลายๆ ค่าไว้ในตัวแปรเดียวผ่าน basic_python/03_loops_lists.py

- **List** คือโครงสร้างข้อมูลที่เก็บค่าได้หลายตัวเรียงตามลำดับในเครื่องหมาย [...]
- เข้าถึงข้อมูลผ่าน **Index** โดยเริ่มนับจากตำแหน่งที่ 0
- สามารถเพิ่มข้อมูลด้วย .append() หรือวนลูปอ่านค่าทีละตัวได้ทันที
- ในโปรเจกต์นี้ เราใช้ List เก็บรายชื่อไฟล์ภาพ และรายการพิกัด Bounding Boxes

```
basic_python/03_loops_lists.py

# 03_loops_lists.py - ลิสต์และการทำซ้ำ Loop
plates = ["1กก1234", "2ขข5678", "3คค9999"]

print("รายการทะเบียนรถในระบบ:")
for p in plates:
    print("🚗 ทะเบียน:", p)
```

Function — การสร้างฟังก์ชันเพื่อเรียกใช้ซ้ำ

จัดกลุ่มคำสั่งให้เป็นระเบียบผ่าน `basic_python/04_functions.py` เพื่อเตรียมเขียนระบบใหญ่

- **ฟังก์ชัน (def)** คือการรวมชุดคำสั่งไว้เป็นบล็อกเดียว สามารถเรียกใช้ซ้ำได้กี่ครั้งก็ได้
- รับค่าเข้าผ่าน **Parameter** และส่งผลลัพธ์กลับด้วยคำสั่ง **return**
- ตัวอย่าง: ฟังก์ชัน `check_plate()` ตรวจสอบว่าป้ายทะเบียนตรงกับ Whitelist หรือไม่
- ช่วยให้โค้ดแบ่งเป็นสัดส่วน ดูแลง่าย และนำไปประกอบเป็น Pipeline ได้สะดวก

```
basic_python/04_functions.py

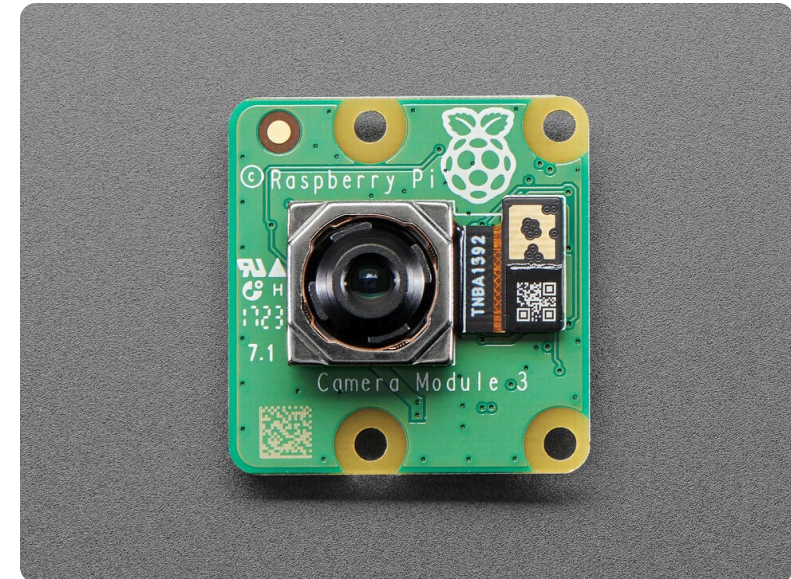
# 04_functions.py - การสร้างฟังก์ชันใช้งานซ้ำ
def check_plate(plate, allowed_list):
    if plate in allowed_list:
        return "อนุญาตผ่าน (ALLOWED)"
    return "ไม่อนุญาต (DENIED)"

whitelist = ["1กก1234", "2ขข5678"]
result = check_plate("1กก1234", whitelist)
print("ผลการตรวจสอบ:", result)
```

รู้จัก Pi Camera Module 3

ฮาร์ดแวร์เซ็นเซอร์กล้องความคมชัดสูงอย่างเป็นทางการสำหรับ Raspberry Pi 5

- เซนเซอร์รับภาพ **Sony IMX708** ความละเอียดสูง **12 ล้านพิกเซล (12MP)**
- ระบบ **Autofocus (PDAF)** โฟกัสวัตถุอัตโนมัติได้อย่างรวดเร็วและแม่นยำ
- เชื่อมต่อผ่านพอร์ต **CSI (Camera Serial Interface) 22-pin** บนบอร์ด Pi 5
- มีเวอร์ชัน Standard FOV (75°) และ Wide FOV (120°) ให้เลือกใช้งาน

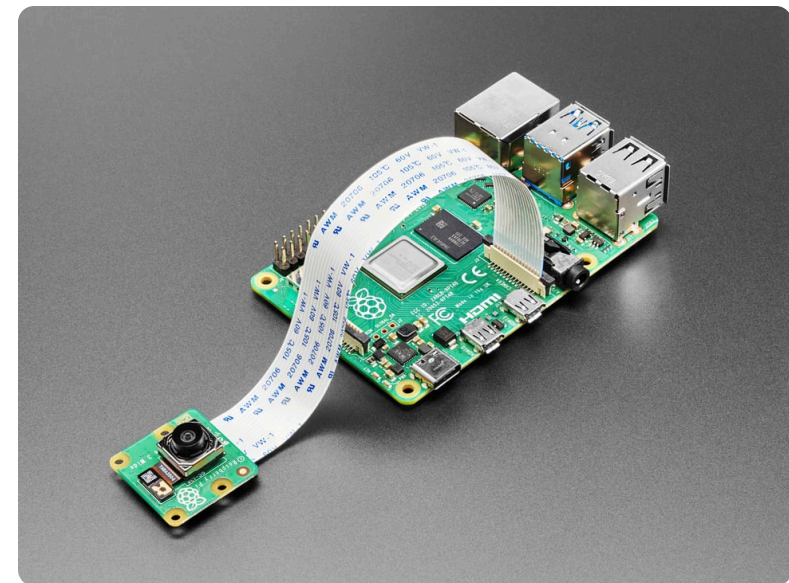


Raspberry Pi Camera Module 3 — เซนเซอร์ Sony IMX708 12MP (ภาพ: Adafruit)

วิธีต่อสาย CSI เข้าบอร์ด Pi 5 อย่างถูกต้อง

ขั้นตอนการติดตั้ง Ribbon Cable เพื่อป้องกันอุปกรณ์เสียหาย

- ⚠️ ปิดเครื่องและถอดสายไฟ Pi 5 ทุกครั้งก่อนเสียบหรือถอดสายกล้อง
- ดึงคลิปล็อกสีดำที่พอร์ต CSI บนบอร์ดขึ้นเบาๆ ในแนวตั้ง
- เสียบสายแพโดยหันแถบโลหะทองแดง (Contact Pins) เข้าหาพอร์ต HDMI
- ดันสายลงไปจนสุดในแนวตรง แล้วกดคลิปล็อกสีดำกลับลงมาให้แน่นสนิท



สาย CSI เชื่อมกล้องเข้ากับบอร์ด Raspberry Pi — ด้านโลหะทองแดงหันเข้าหาพอร์ต HDMI (ภาพ: Adafruit)

เปิดใช้งานและทดสอบกล้องครั้งแรก

ตรวจสอบการเชื่อมต่อฮาร์ดแวร์กล้องผ่าน libcamera บน Raspberry Pi OS

- เปิด Terminal แล้วทดสอบด้วยคำสั่ง `rpicas-hello` เพื่อเช็การเชื่อมต่อ
- ถ้ากล้องต่อถูกต้อง จะปรากฏหน้าต่าง **Preview** ภาพสดขึ้นมาทันที
- ทดลองสั่งถ่ายภาพนิ่งและบันทึกลงไฟล์ด้วยคำสั่ง:

```
rpicas-still -t 2000 -o test.jpg
```

```
Terminal - rpicas-hello

pi@raspberrypi:~ $ rpicas-hello
[0:00:01.204] INFO Camera module: imx708 [4608x2592]
[0:00:01.312] INFO Preview window opened successfully
✓ กล้องทำงานปกติ กำลังแสดงภาพสดบนหน้าจอ ...
```

ถ่ายภาพนิ่งด้วย picamera2 (capture.py)

เขียนโปรแกรม Python ควบคุมกล้องผ่าน license_plate_capture/capture.py

- **picamera2** คือไลบรารีอย่างเป็นทางการสำหรับการควบคุมกล้อง Pi Camera ผ่าน Python
- ตั้งความละเอียดภาพที่เหมาะสม เช่น **2304 × 1296 พิกเซล**
- หน่วงเวลา **Warmup 2 วินาที** เพื่อให้กล้องปรับสมดุลแสง (Auto Exposure) และโฟกัส
- บันทึกภาพลงไฟล์เดอร์
license_plate_crop/input_images/ โดยตรงเพื่อส่งต่อให้ AI

```
license_plate_capture/capture.py

from picamera2 import Picamera2
import time

cam = Picamera2()
cfg = cam.create_still_configuration(main={"size": (2304,
cam.configure(cfg)
cam.start()
time.sleep(2)  # รอปรับแสง & โฟกัส

cam.capture_file("../license_plate_crop/input_images/car.jpg")
cam.stop()
print("ถ่ายภาพและบันทึกเรียบร้อยแล้ว!")
```

ถ่ายภาพผ่าน Web GUI (capture_gui.py)

หน้าเว็บ Streamlit สำหรับดูตัวอย่างภาพสดและสั่งถ่ายภาพด้วยคลิกเดียว

- รันสคริปต์ `./startCaptureGUI.sh` เพื่อเปิดหน้าเว็บที่ `http://localhost:8501`
- ปุ่ม "ดูตัวอย่าง" (`cam.capture_array()`) — ดึงภาพปัจจุบันมาแสดงเพื่อใช้คอมมูมกล้องโดยยังไม่บันทึก
- ปุ่ม "ถ่ายภาพและบันทึก" (`cam.capture_file()`) — บันทึกภาพความละเอียดเต็มพร้อม timestamp
- เปิดกล้องครั้งเดียวเก็บใน `st.session_state` เพื่อความรวดเร็วและไม่กระตุก

```
license_plate_capture/capture_gui.py

import streamlit as st
from picamera2 import Picamera2

col1, col2 = st.columns(2)
with col1:
    if st.button("ดูตัวอย่าง"):
        frame = camera.capture_array()
        st.image(frame, caption="มุมมองกล้องสด")
with col2:
    if st.button("ถ่ายภาพและบันทึก", type="primary"):
        camera.capture_file(save_path)
        st.success("บันทึกเข้า input_images สำเร็จ!")
```

Python + OpenCV เบื้องต้น

เครื่องมือมาตรฐานสากลสำหรับการจัดการและประมวลผลข้อมูลภาพ (Computer Vision)

- **cv2.imread() / cv2.imshow()** โหลดและแสดงผลภาพเป็น NumPy Array
- **Grayscale** (cv2.COLOR_BGR2GRAY) ลด 3 แชนแนลสีเหลือ 1 แชนแนลเพื่อความเร็ว
- **Resize & Crop** ปรับขนาดและตัดเฉพาะขอบเขตพื้นที่ที่สนใจ (ROI)
- **Canny Edge & Thresholding** หาขอบวัตถุและแปลงภาพเป็นขาวดำคมชัด



Contour Detection แบบเดิม & สรุปวันที่ 1

ทดลองหากรอบป้ายทะเบียนด้วยวิธีเขียนกฎ (Rule-based) และข้อจำกัดที่พบ

- ลองใช้ **cv2.findContours()** หาสี่เหลี่ยมที่มีอัตราส่วนกว้าง:ยาว ใกล้เคียงป้ายทะเบียน
- จุดอ่อนของวิธีนี้: เมื่อแสงเปลี่ยน มุมป้ายเอียง มีเงา หรือพื้นหลังซับซ้อน จะตรวจจับหลุดทันที
- **สรุปวันที่ 1:** ได้ฮาร์ดแวร์ RPi 5 + กล้อง + ภาษา Python + พื้นฐานการจัดการภาพ
- **โจทย์ใหญ่ของวันที่ 2:** เราจะนำ **AI Deep Learning (YOLOv8)** มาแก้ปัญหานี้ได้อย่างไร?

Contour Rule: แสงเปลี่ยน / มืด ✗ หลุด

Contour Rule: มุมกล้องเอียง ✗ หลุด

Contour Rule: ลายเส้นหนักรถ ✗ ผิดตำแหน่ง

↓ ทางออกในวันที่ 2

 ใช้ YOLOv8 AI Model ตรวจจับแม่นยำทุกสภาพแสง

วันที่ 2



YOLOv8n + Thai Tesseract OCR + Data Matching & Web GUI

🎯 นำ AI Deep Learning มาตรวจจับป้ายทะเบียน + อ่านอักขระไทย + แมตซ์ 77 จังหวัด + ประกอบเป็นระบบ GUI เติมรูปแบบ

บทวน Day 1 + ทำไมต้องใช้ Deep Learning

เปรียบเทียบการเขียนโปรแกรมแบบเดิม (Rule-based) กับการเรียนรู้ของโมเดล AI (Deep Learning)

- **Rule-based (เมื่อวาน):** มนุษย์ต้องเขียนเงื่อนไขทุกอย่างเอง ถ้ารถเปลี่ยนมุม แสงเปลี่ยน กฎจะพังทันที
- **Deep Learning (วันนี้):** คอมพิวเตอร์เรียนรู้ลักษณะของ "ป้ายทะเบียน" จากภาพตัวอย่างนับพันภาพ
- ทนทานต่อสภาพแวดล้อมจริง: ทั้งกลางวัน กลางคืน แสงสะท้อน และมุมกล้องที่เอียง

Rule-based ❌

เขียนกฎเองด้วยมือ
เผลอเผลอ แสงเปลี่ยนก็พัง

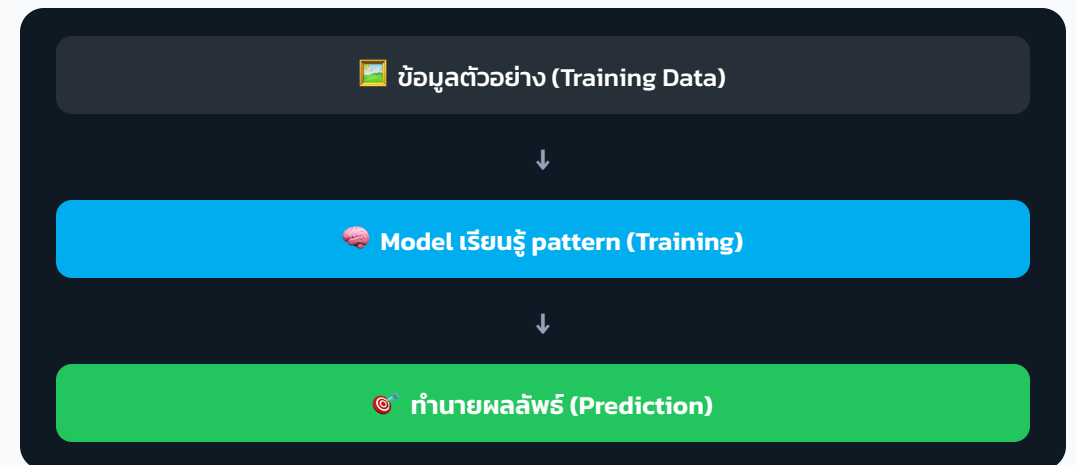
YOLOv8 Deep Learning ✔️

เรียนรู้จากชุดข้อมูลจริง
แม่นยำ ทนทาน เสถียรสูง

หลักการทำงานของ Machine Learning

โมเดล AI "เรียนรู้" จากตัวอย่างข้อมูล แทนที่มนุษย์จะเขียนกฎเองทุกขั้นตอน

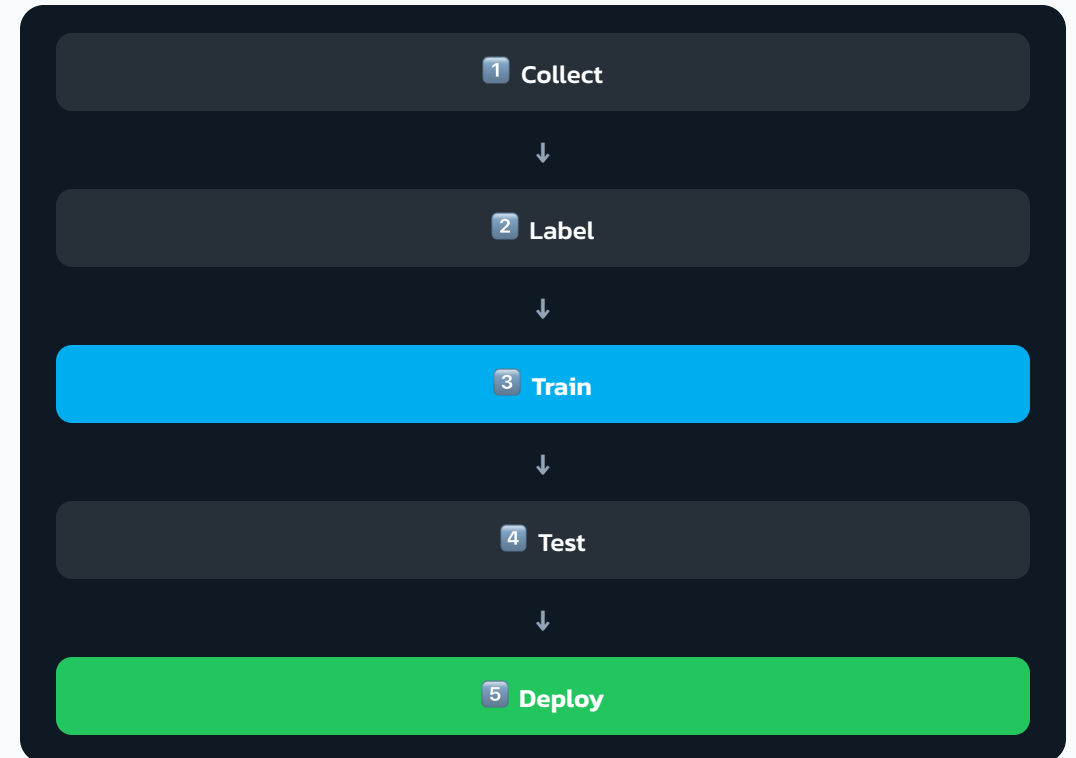
- **เขียนโปรแกรมแบบเดิม:** มนุษย์กำหนดกฎ (if-else) เองทุกกรณี เหมือนที่เขียนเมื่อวาน
- **Machine Learning:** ป้อน "ตัวอย่างข้อมูล" จำนวนมากให้โมเดล แล้วให้มันหา pattern เอง โดยไม่ต้องเขียนกฎ
- **Model** คือฟังก์ชันคณิตศาสตร์ขนาดใหญ่ ข้างในมีค่าพารามิเตอร์ (Weights) นับล้านค่าที่ปรับได้
- **Training** คือการค่อยๆ ปรับค่าพารามิเตอร์เหล่านั้น จนโมเดลทำนายผลจากตัวอย่างได้แม่นยำขึ้นเรื่อยๆ



5 ขั้นตอนของ Machine Learning Workflow

ทุกโมเดล AI ไม่ว่าเล็กหรือใหญ่ ก็ผ่านกระบวนการเดียวกันนี้ทั้งหมด รวมถึง YOLOv8 ที่จะเทรนกันวันนี้

- **1. Collect** — เก็บข้อมูลตัวอย่าง เช่น ภาพป้ายทะเบียนหลายๆ รูป
- **2. Label** — ติดป้ายกำกับ/แบ่งคลาสให้ข้อมูล เช่น วาดกรอบตำแหน่งป้ายทะเบียนในแต่ละภาพ
- **3. Train** — ให้โมเดลเรียนรู้จากข้อมูลที่ label ไว้ ปรับพารามิเตอร์ซ้ำๆ หลายรอบ (epochs)
- **4. Test** — ทดสอบกับข้อมูลที่โมเดลไม่เคยเห็นมาก่อน วัดความแม่นยำ
- **5. Deploy** — นำโมเดลไปใช้งานจริง เช่น ใส่ในกล้อง Pi Camera ของเรา



Workshop: สร้างโมเดล AI ของตัวเองด้วย Teachable Machine

ลองฝึกโมเดล Image Classification แบบไม่ต้องเขียนโค้ดเลย ผ่านเว็บเบราว์เซอร์ ก่อนไปเรียน YOLOv8

- เปิด **teachablemachine.withgoogle.com** เครื่องมือฟรีจาก Google สำหรับฝึก AI แบบลากวาง
- เลือก **Get Started** → **Image Project** → **Standard image model**
- สร้างอย่างน้อย 2 Class แล้วเก็บภาพตัวอย่างผ่านเว็บแคม เช่น "ยกนิ้วโป้ง" กับ "กำหมัด"
- กด **Train Model** แล้วดูผลทำนายแบบ real-time ได้ทันทีในช่อง Preview
- นี่คือการ **เดียวกัน** กับที่ YOLOv8 ใช้เทรนตรวจจับป้ายทะเบียนที่จะเรียนต่อจากนี้ เพียงแต่ Teachable Machine ทำให้เห็นภาพขั้นตอน Collect → Train → Test ได้ง่ายและเร็วกว่ามาก



YOLOv8 คืออะไร & การเตรียม Dataset / Train

โครงสร้างการทำงานของโมเดล Object Detection ผ่าน license_plate_crop/train_model1.py

- **YOLO (You Only Look Once):** โมเดล Object Detection ที่เร็วและแม่นยำสูง เหมาะกับบอร์ด Edge AI
- **Dataset:** ดาต้าโฮสต์ชุดภาพป้ายทะเบียนจาก **Roboflow Universe** (ฟอร์แมต YOLOv8) วางใน dataset/
- สคริปต์ train_model1.py สั่งเทรนโมเดล และได้น้ำหนักที่ดีที่สุด (best.pt)
- บันทึกเป็นไฟล์โมเดล models/model1_plate_detect.pt พร้อมนำไปใช้งานทันที

```
license_plate_crop/train_model1.py

from ultralytics import YOLO

# โหลดโมเดลตั้งต้น yolov8n.pt
model = YOLO("yolov8n.pt")

# สั่งเทรนด้วย dataset ป้ายทะเบียน
model.train(
    data="dataset/data.yaml",
    epochs=50,
    imgsz=640
)

# ผลลัพธ์: models/model1_plate_detect.pt
```

โครงสร้างโปรเจกต์จริง & Edge-AI Environment

แผนผังโฟลเดอร์และหน้าที่ของแต่ละสคริปต์ใน ComVision_workshop

- โครงสร้างแบ่งเป็น 5 โมดูลอิสระ และ 1 สคริปต์กลางสำหรับรันครบวงจร
- **license_plate_capture** — ถ่ายภาพจากกล้อง Pi Camera
- **license_plate_crop** — ตรวจจับพิกัดและ Crop ป้ายด้วย YOLOv8
- **license_plate_ocr** — อ่านตัวอักษรไทยด้วย Tesseract OCR
- **license_plate_match** — ทำความสะอาดข้อความและ Match 77 จังหวัด
- **license_plate_gui** — หน้าเว็บ Streamlit สำหรับผู้ใช้งาน

ComVision_workshop/

- | **basic_python/** (01-04.py ตัวอย่างไวยากรณ์)
- | **license_plate_capture/** (capture.py, capture_gui.py)
- | **license_plate_crop/** (detect_and_crop.py, models/)
- | **license_plate_ocr/** (ocr_plate.py, output_text/)
- | **license_plate_match/** (clean_and_match.py, provinces_th.txt)
- | **license_plate_gui/** (app.py, startGUI.sh)
- | **run_pipeline.py** (รันทุกโมดูลต่อเนื่องในคำสั่งเดียว)
- | **setup.sh** (สคริปต์ติดตั้งระบบ & venv อัตโนมัติ)

ตรวจจับและ Crop ป้ายทะเบียน (detect_and_crop.py)

ใช้โมเดล YOLOv8 หาพิกัดป้ายทะเบียนและตัดภาพเฉพาะส่วนป้ายอัตโนมัติ

- โหลดโมเดล `models/model1_plate_detect.pt`
- รับ Predict ภาพจาก `input_images/` ด้วยค่าความมั่นใจ `conf = 0.5`
- ได้พิกัดกรอบ Bounding Box: `[x1, y1, x2, y2]`
- ตัดเฉพาะบริเวณป้าย (`img.crop()`) และบันทึกลง `output_crops/`

```
license_plate_crop/detect_and_crop.py

from ultralytics import YOLO
from PIL import Image

model = YOLO("models/model1_plate_detect.pt")
results = model.predict(source=str(img_path), conf=0.5)
boxes = results[0].boxes.xyxy.tolist()

orig_img = Image.open(img_path)
for idx, (x1, y1, x2, y2) in enumerate(boxes):
    crop_img = orig_img.crop((x1, y1, x2, y2))
    crop_img.save(f"output_crops/{img_path.stem}_plate_{idx}.jpg")
```

อ่านอักขระไทยด้วย Tesseract (ocr_plate.py)

เทคนิคการ Preprocess ภาพป้ายทะเบียนเพื่อเพิ่มความแม่นยำในการอ่านข้อความ

- **1. Inner Crop:** ตัดขอบบน-ล่าง 18% และซ้าย-ขวา 10% เพื่อตัดขอบดำและรอยสกปรกที่รบกวน OCR
- **2. Grayscale & 3x Upscaling:** แปลงขาวดำและขยายภาพ 3 เท่า (INTER_CUBIC) ให้ฟอนต์ไทยคมชัด
- **3. Tesseract OCR:** กำหนดภาษา lang="tha+eng" และโหมด --psm 11 (Sparse Text)
- บันทึก Raw Text ที่อ่านได้ลงใน
license_plate_ocr/output_text/plate_texts.csv

```
license_plate_ocr/ocr_plate.py

# 1. ตัดขอบดำ/รอยสกปรกออก
pad_y, pad_x = int(h * 0.18), int(w * 0.10)
inner = img[pad_y:h-pad_y, pad_x:w-pad_x]

# 2. Grayscale + ขยาย 3 เท่า
gray = cv2.cvtColor(inner, cv2.COLOR_BGR2GRAY)
resized = cv2.resize(gray, (w*3, h*3), interpolation=cv2.INTER_CUBIC)

# 3. อ่านข้อความไทย + อังกฤษ
text = pytesseract.image_to_string(resized, lang="tha+eng")
# ผลลัพธ์: "กค 9417 | กรุงเทพมหานคร"
```

Data Cleaning & Matching 77 จังหวัด (clean_and_match.py)

ทำความสะอาดข้อความ แยกเลขทะเบียน และแก้คำสะกดเพี้ยนด้วย Fuzzy Matching

- **1. Cleaning:** ใช้ Regular Expression ตัดสัญลักษณ์ขยะ: `([^ก-๐0-9\s])`
- **2. แยกเลขทะเบียน:** ค้นหาแพทเทิร์น `[ก-ฮ]{1,3}\s?\d{1,4}` ได้เลขทะเบียน เช่น "กค 9417"
- **3. Fuzzy Matching จังหวัด:** เทียบชื่อจังหวัดกับ `provinces_th.txt` (77 จังหวัด) ด้วย `difflib`
- บันทึกผลลัพธ์ลง
`license_plate_match/output_data/plates_matched.csv`

```
license_plate_match/clean_and_match.py

# 1. Regex กรองเฉพาะภาษาไทยและตัวเลข
clean_text = re.sub(r"[^ก-๐0-9\s]", "", raw_text)

# 2. แยกเลขทะเบียนไทย
plate_num = re.search(r"[ก-ฮ]{1,3}\s?\d{1,4}", clean_text)

# 3. Fuzzy Match เทียบ 77 จังหวัด
match = difflib.get_close_matches(seg, provinces, n=1, cutoff=0.6)
score = difflib.SequenceMatcher(None, seg, match[0])
# ผลลัพธ์ CSV: Thai_car_2_plate_0.jpg, กค 9417, กรุงเทพฯ
```

รวม Pipeline ทั้งระบบ (CLI & Streamlit Web GUI)

ประกอบทุกโมดูลเข้าด้วยกัน ทำงานได้ทั้งผ่าน Command Line และหน้าเว็บ Interactive Dashboard

1. กล้อง (Capture) → 2. YOLOv8 Crop → 3. Tesseract OCR → 4. Clean & Match → 5. Streamlit Web GUI

CLI Mode: run_pipeline.py

```
pi@raspberrypi:~$ python run_pipeline.py
=== ขั้นตอนที่ 1: ตรวจจับ + Crop ป้ายทะเบียน ===
บันทึกแล้ว: Thai_car_2_plate_0.jpg
=== ขั้นตอนที่ 2: OCR อ่านข้อความป้ายทะเบียน ===
Thai_car_2_plate_0.jpg: กค 9417 | กรุงเทพมหานคร
=== ขั้นตอนที่ 3: Data Cleaning & Matching ===
✓ สำเร็จ: เลขทะเบียน='กค 9417' จังหวัด='กรุงเทพมหานคร' (1.0)
```

Web GUI Mode: license_plate_gui/app.py

```
pi@raspberrypi:~$ ./license_plate_gui/startGUI.sh
เปิดหน้าเว็บที่ http://localhost:8501
• อัปโหลดภาพรถยนต์ผ่าน Browser
• กดปุ่ม "ประมวลผล" รันทั้ง 3 ขั้นตอนอัตโนมัติ
• แสดงภาพป้ายที่ตัดได้ + เลขทะเบียน + จังหวัด + Score
✓ ใช้งานง่าย ไม่ต้องพิมพ์คำสั่งใน Terminal
```

ทดสอบกับรถจริง/ภาพจริง & การจูนระบบ (Tuning)

นำระบบไปทดสอบกับภาพตัวอย่างในโฟลเดอร์ photo/ และรถยนต์จริงหน้างาน

- ทดสอบกับ Dataset จริง: รับภาพจาก
photo/Thai_car_1.jpeg และ
photo/Thai_car_2.jpeg
- จูนค่าตัวแปรสำคัญ (Hyperparameters):
 - CONF_THRESHOLD = 0.5 — ปรับความมั่นใจในการตรวจจับป้าย
 - CROP_RATIO_Y / X = 0.18 / 0.10 — ปรับขนาดการตัดขอบดำและรอยสกปรก
 - MATCH_CUTOFF = 0.6 — ปรับความยืดหยุ่นในการค้นหาชื่อจังหวัด
- แก้ปัญหาหน้างาน: ปรับมุมกล้อง หลีกเลียงแสงสะท้อน และตั้งระยะห่างให้เหมาะสม



ทดสอบอ่านป้ายทะเบียนรถยนต์จริงในสภาพแสงและมุมมองต่างๆ (ภาพ: Wikimedia Commons)

นำเสนอผลงาน + Q&A + แนวทางต่อยอด

สรุปความสำเร็จตลอดการอบรมเชิงปฏิบัติการ 2 วันเต็ม

- **Student Showcase:** แต่ละกลุ่มนำเสนอผลการทดสอบระบบและปัญหาที่พบพร้อมวิธีแก้
- **แนวทางต่อยอดสู่โครงการจริง:**
 - ต่อดีเลย์/มอเตอร์สั่งเปิด-ปิดไม้กั้นอัตโนมัติ (**Smart Barrier Gate**)
 - ส่งการแจ้งเตือนและรูปภาพเข้า **LINE Notify / Telegram Bot**
 - บันทึกสถิติเวลาเข้า-ออกรถยนต์ลงในฐานข้อมูล **SQL / Cloud Database**
- 🎉 **ช่วงถาม-ตอบ Q&A + ถ่ายภาพหมู่ร่วมกันเพื่อปิดคอร์สอย่างสมบูรณ์**



ขอแสดงความยินดีกับทุกคน!

Cytron Technologies · AI Camera Workshop